# A Preregistered, Artifact-Backed Paired Evaluation of a Generate–Validate–Repair Pipeline for Intent→Configuration NetOps Tasks

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

***Abstract*—We preregistered and executed GVR-ProtocolEval-2026-PR1, a deterministic, artifact-archived paired experiment comparing LLM-only generation and a guarded generate–validate–repair (GVR) evaluation pipeline on 200 stratified intent→configuration NetOps tasks in a Dockerized Mininet emulator with a deterministic validator. Under the frozen preregistration the same single LLM candidate was reused for both arms (generation_semantics = "shared_initial_candidate"). All confirmatory episodes were verifier-accepted: baseline 200/200, guarded 200/200 (n11=200, n10=0, n01=0, n00=0). Paired difference = 0.00; 95% bootstrap percentile CI = [0.00, 0.00] (10,000 resamples, seed = 1729); McNemar exact p = 1.0. Because the preregistered semantics produced one shared generation per task no repair loop was exercised in the confirmatory runs; the observed null is therefore an artifact-grounded boundary condition. We publish the complete preregistration, raw model outputs, provider traces, deterministic validator scoring JSONs (representative: execution/scoring/task-000001-guarded.json SHA-256 9235bce7875c1717dff5ac6be49c5769929b4e8509f5df404127efe9d6ee938f), execution manifest (execution/execution_manifest.jsonl SHA-256 d76af611272bd8e8b3a94290efecd6ecf634c5f3e36c2450c4245985e96931fb), and analysis artifacts (analysis/paired_contingency_table.csv SHA-256 7bc1bd2a42b5ac3f7adb61db47cf8dbe0472f678032abcd1e7c44f92ea2de71c) so the run is deterministically reproducible. We report artifact-grounded diagnostics and prescribe preregistered follow-ups (independent draws per arm, expanded validator coverage, adversarial templates) that the archived bundle already enables.**

## I. Introduction

Automating the translation of high-level intent into device configurations is an operationally valuable but safety-critical NetOps task: incorrect automation can produce reachability loss, policy violations, and outages. Architectures that pair LLM-driven generation with deterministic validators and bounded repair loops (generate→validate→repair, GVR) aim to improve operational trustworthiness by rejecting unsafe candidates and invoking constrained repairs. Prior work has produced benchmarks and surveys for LLM-enabled NetOps (NetConfEval, OpsEval, NetEval) and proposals for guarded agentic NetOps pipelines, but few studies present a fully preregistered, deterministic, artifact-archived paired evaluation where every principal number is traceable to immutable files. This paper contributes a methodologically strict, fully archived demonstration run: (1) a machine-readable preregistration (GVR-ProtocolEval-2026-PR1; SHA-256 d913b4167cc6aa85b283e0703e0bfff0c3e8864405f5159d9a1a1cf5fc3fafdc), (2) an executed paired experiment on 200 stratified tasks with deterministic scoring, (3) complete published artifacts (model calls, validator traces, scoring JSONs, execution manifest) enabling deterministic reproduction (execution/execution_manifest.jsonl SHA-256 d76af611272bd8e8b3a94290efecd6ecf634c5f3e36c2450c4245985e96931fb), and (4) an evidence-grounded account of why the confirmatory result is a boundary condition and what preregistered follow-ups are supported by the archive.

## II. Related Work

Recent NetOps and AIOps evaluations explore LLMs for intent→configuration and operational tasks: NetConfEval examined intent→configuration utility and prototypes (Wang et al., NetConfEval), OpsEval produced a broad Ops benchmark and leaderboard for AIOps tasks (Liu et al., OpsEval), and NetEval assessed NetOps capabilities across many models (Miao et al., NetEval). Work on guarded execution, generate→validate→repair, and verifier-assisted generation has parallels in program repair and verifier-guided code synthesis literature. Our contribution is methodological: we couple strict preregistration, a deterministic validator, and full artifact publication so that every principal number maps to archived files (examples in cited_record_ids). This makes an otherwise surprising null reproducible and diagnostically traceable.

## III. Methodology

Preregistration and frozen design. We created a machine-readable preregistration (GVR-ProtocolEval-2026-PR1; SHA-256 d913b4167cc6aa85b283e0703e0bfff0c3e8864405f5159d9a1a1cf5fc3fafdc) that froze the primary estimand (paired difference in binary operational-failure indicator), the confirmatory paired design (200 tasks stratified across 5 protocol groups × 2 interaction modalities), the exact model family (gpt-5-mini recorded in execution/model_configuration.json SHA-256 a40e96e212ab87da251ac0d6dbb75bc543afa13438b5a6b9ebd073597ffdd82), contamination thresholds (token overlap >=80% → exclude), bootstrap parameters (10,000 resamples, seed=1729), retry rules (infrastructure recovery only), and generation semantics. Generation semantics (frozen): generation_semantics = "shared_initial_candidate" — the same single LLM draw for each task was reused under both arms so the confirmatory

contrast tests verifier acceptance of that candidate rather than repair effectiveness. This choice was preregistered and archivally recorded; the exact initial master prompt is archived as provenance/master_prompt.txt (SHA-256 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf15820bc30b15a). Task bank and emulator. A deterministic task generator (deterministic_netops_task_generator_v5) produced a confirmatory bank of 200 intent→configuration tasks (≈20 per protocol×modality cluster) plus 50 exploratory tasks. Each task manifest encodes: initial state, required final state, explicit workflow and safety constraints (allowed_touched_fields, must_be_down_for_fields, minimum_active_in_groups), and the canonical required command sequence. The experiments ran in a Dockerized Mininet emulator under a deterministic harness; all task manifests and hashes are archived at execution/task_manifest.jsonl (SHA-256 c1bd4a096ee4d6ce21266f57b01a24a8aa412f7e6722487840c6a955ed900d32). Deterministic validator and scoring. We used a deterministic validator (deterministic_netops_validator_v5) implementing parsing, dependency checks, and final-state simulation; it produces a scoring JSON that includes validation_before/after snapshots and normalized_configuration. Representative scoring JSONs are archived (e.g., execution/scoring/task-000001-guarded.json SHA-256 9235bce7...). Contamination checks and provenance. The preregistered contamination rule (token overlap >=80% against curated vendor/RFC corpus) was run; results are archived (analysis/contamination_summary.csv SHA-256 389227f0...). Full provider traces and call caches are archived (execution/provider_calls/ and execution/call_cache/ entries; manifest contains SHA-256 list). Statistical analysis plan. Primary analysis: paired mean difference in binary operational-failure (fail = reachability loss or policy violation), estimated by bootstrap percentile CI (10,000 resamples, seed 1729) and tested with exact McNemar two-sided. Multiplicity: single primary confirmatory test only; subgroup analyses labeled secondary/exploratory. Reproducibility guarantees. All code, generation seeds, model configuration, provider traces, raw responses, validator scoring JSONs, and analysis scripts are archived and SHA-256 hashed so the entire pipeline is deterministically replayable (execution/execution_manifest.jsonl SHA-256 d76af611...). Representative artifact paths and SHA-256 values are cited throughout.

## IV. Results

Execution and raw artifacts. Planned episodes = 400 (200 tasks × 2 arms). Completed episodes = 400; failed episodes = 0. Model calls used = 200 (one shared generation per task under the preregistered shared_initial semantics). All raw artifacts and hashes are archived: execution/execution_manifest.jsonl (SHA-256 d76af611272bd8e8b3a94290efecd6ecf634c5f3e36c2450c4245985e9603fba). Confirmatory contingency. The deterministic scoring JSONs produce the paired contingency (analysis/paired_contingency_table.csv SHA-256 7bc1bd2a42b5ac3f7adb61db47cf8dbe0472f678032abcd1e7c44f92ea2de71c): n11=200, n10=0, n01=0, n00=0. Primary inferential results. Paired difference (guarded - baseline) = 0.00; 95% bootstrap percentile CI = [0.00, 0.00] (10,000 resamples, seed = 1729); McNemar exact two-sided p = 1.0. Representative scoring artifacts. Example task and scoring artifacts (archived) that illustrate validator traces and final_state snapshots: execution/scoring/task-000001-guarded.json (SHA-256 9235bce7...); execution/scoring/task-000002-guarded.json (SHA-256 25e2c0c7...); execution/scoring/task-000003-guarded.json (SHA-256 20e953b1...). Each scoring JSON contains parsed_command_count, required_command_count, validation_before/after states, and normalized_configuration — enabling command-level reconstruction of why a candidate passed. Contamination and checks. The preregistered contamination scan found no confirmatory tasks to exclude under the >=80% token overlap rule (analysis/contamination_summary.csv SHA-256 389227f0...). Diagnostics founded on artifacts. Because the preregistered semantics reused the same LLM candidate under both arms, no repair loop was applied (repair_applied=false in scoring JSONs). The artifact set supports three artifact-backed observations: (A) task templates were solvable by straightforward command sequences (task manifests archived at execution/task_manifest.jsonl SHA-256 c1bd4a09... show required_sequence fields), (B) the deterministic validator's rule set matched those solvable templates (scoring JSONs show parsed_command_count == required_command_count and valid=true), and (C) the contamination scans recorded no high-overlap candidates. Together these archived facts explain the observed 100% verifier acceptance as a reproducible boundary condition of the frozen generation semantics, not as an evaluation of active repair efficacy.

## V. Discussion

What the confirmatory null means. The confirmatory paired test measured whether a single shared LLM candidate per task would be accepted by the deterministic validator under two conditions; it did not exercise a repair loop because no repairs were invoked (shared-candidate semantics). Observing baseline=100% is therefore a valid, reproducible outcome of the preregistered design but it does not falsify the hypothesis that active GVR repair loops can reduce failures when baseline error rates are nonzero. Artifact-based diagnostics. The complete artifact bundle permits deterministic diagnosis: (1) contamination_summary.csv (SHA-256 389227f0...) shows no tasks excluded for overt token overlap under the preregistered threshold; (2) scoring JSONs (e.g., execution/scoring/task-000001-guarded.json SHA-256 9235bce7...) include validator traces showing required_command_count == parsed_command_count and valid==true for representative tasks; (3) task manifest records demonstrate the confirmatory templates were constructed to be solvable by the canonical sequences listed in required_sequence fields. These artifacts support a parsimonious account: for the sampled synthetic templates the LLM

reliably produced safe command sequences and the validator accepted them. Design lessons and recommended preregistered follow-ups. The run exposes how preregistration + full archival converts an unexpected null into an actionable diagnostic. We propose three preregistered follow-ups (all feasible using the archived harness and task generator): (A) Independent draws per arm: set generation_semantics = "independent_generation_per_arm" so baseline and guarded receive independent LLM samples and allow repairs to run in guarded; recompute paired outcomes under identical validator settings. (B) Stronger validator coverage: extend deterministic_netops_validator_v5 with timing-sensitive reachability probes, multi-step transient checks, and richer policy assertions; rerun the confirmatory bank to increase construct difficulty. (C) Adversarial templates: modify the task generator to produce templates violating simple presence checks (e.g., required reorderings or vendor-specific command variants) to evaluate where repairs are needed. Each follow-up should be preregistered and archived; the current bundle contains the exact seeds, generator, validator, and emulator needed to run them deterministically. Threats to validity. Internal: single-candidate semantics, fixed seeds, and deterministic validator rules can produce boundary results; external: Mininet emulation and synthetic templates limit production generalizability; construct: validator rule set may not capture all operational hazards (firmware bugs, device timing); conclusion: observed null invalidates the preregistered power assumption (see next).

## VI. Limitations

- Emulation and scope: confirmatory experiments used a Dockerized Mininet emulator and synthetic vendor-like templates; results do not directly generalize to heterogeneous production devices, firmware versions, or timing side effects.
- Shared-candidate semantics: preregistration froze generation_semantics = "shared_initial_candidate" so each task used one LLM draw reused under both arms; consequently no repair loop was exercised in confirmatory episodes and the run does not test active repair efficacy.
- Validator coverage: deterministic_netops_validator_v5 implements a finite syntactic and dependency rule set; operational hazards outside those rules (e.g., device-specific race conditions or firmware idiosyncrasies) are not covered.
- Power-plan mismatch: the preregistered power calculation assumed a nonzero baseline failure rate; observing baseline=100% invalidates that assumption and the planned MDE cannot be assessed from this confirmatory run.
- Contamination detection limits: the preregistered token-overlap check (>=80% threshold) flagged no confirmatory exclusions, but nondiscoverable model pretraining overlap with vendor examples cannot be ruled out wholly.
- External generalizability: the artifact-grounded boundary condition observed here does not imply GVR provides no benefit in harder, production, or adversarial settings; it only documents that under the frozen generation semantics and solvable templates the LLM candidates satisfied the deterministic validator.

## VII. Conclusion

We preregistered, executed, and fully archived a deterministic paired experiment comparing LLM-only generation and a guarded GVR pipeline on 200 synthetic intent→configuration tasks. Under the frozen shared-candidate semantics every confirmatory candidate was accepted (n11=200), producing a paired difference of 0.00 (95% bootstrap CI [0.00,0.00], McNemar p=1.0). This outcome is a reproducible boundary condition traceable to archived task manifests, model responses, validator traces, and contamination scans. The scientific value is methodological: strict preregistration plus full artifact publication allows precise, command-level forensic diagnosis and supports immediately executable, preregisterable follow-ups that genuinely test active repair. We therefore release the full artifact bundle and recommend the field adopt this reproducibility discipline for future guard/repair evaluations so that outcomes (including nulls) remain auditable at the command and validator trace level.

## Disclosure Statement

Mandatory Disclosure Statement (CNSM Agentic AI Researcher track). - LLM(s) used: gpt-5-mini (declared_model_version; see execution/model_configuration.json SHA-256 a40e96e212ab87da251ac0d6dbb75bc543afa13438b5a6b9ebd073597ffdd82 - Orchestration/adapter framework: hosted_netops_gvr_v1 (frozen adapter family used to run the preregistered experiment). - Exact initial master prompt immutable archived reference (required by CNSM): provenance/master_prompt.txt, SHA-256 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15 (The preregistration required providing an immutable archived path and SHA-256; that path and SHA-256 are included above and are present in the supplied evidence bundle.) - Research Director role: a human Research Director selected the NetOps topic family and approved preregistration constraints prior to the autonomy lock. After the autonomy lock the autonomous system performed literature discovery, preregistration authoring, experiment design, code generation, execution, analysis, manuscript drafting, autonomous peer review, and revision without human manual editing of technical content, quantitative results, validator outputs, or manuscript technical claims. No human technical edits to those items were made post-lock. - Preregistration/freeze procedure: GVR-ProtocolEval-2026-PR1 (machine-readable preregistration JSON; SHA-256 d913b4167cc6aa85b283e0703e0bfff0c3e8864405f5159d9a1a1cf5fc3fafdc) The preregistration froze: the confirmatory paired design (200 tasks), primary estimand (paired binary difference),

exact bootstrap and test parameters (10,000 resamples, seed=1729; exact McNemar test), generation_semantics = "shared_initial_candidate", contamination thresholds (token overlap >=80%), retry rules (infrastructure-only), and all major analysis decisions. The preregistration preceded execution and is archived in the evidence bundle. - Frameworks, tools, and artifacts: deterministic task generator (deterministic_netops_task_generator_v5), deterministic validator (deterministic_netops_validator_v5), Dockerized Mininet emulator harness, and deterministic analysis scripts; the full execution manifest and artifacts are archived (execution/execution_manifest.jsonl SHA-256 d76af611272bd8e8b3a94290efecd6ecf634c5f3e36c2450c4245985a063ff6b). Representative scoring JSONs with validator traces are archived (e.g., execution/scoring/task-000001-guarded.json SHA-256 9235bce7875c1717dff5ac6be49c5769929b4e8509f5df404127efe9d6ee938f). All raw responses, provider traces and call cache entries are archived under execution/provider_calls/ and execution/call_cache/ and referenced in the execution manifest with SHA-256 hashes. - Contamination policy and checks: preregistered automated token-overlap checks against a curated vendor/RFC corpus; items with token overlap >=80% would be excluded from the confirmatory analysis. The run produced no confirmatory exclusions; the contamination scan artifact is archived at analysis/contamination_summary.csv (SHA-256 389227f0d1e81239c7498abd4944a04c59c92c5fa042b990e332796cd61c3222). - Autonomous analysis procedures: bootstrap percentile CI (10,000 resamples, seed=1729) and exact McNemar two-sided test were preregistered and executed deterministically; principal analysis artifacts are archived (analysis/paired_contingency_table.csv SHA-256 7bc1bd2a42b5ac3f7adb61db47cf8dbe0472f678032abcd1e7c44f92ea2de71c; analysis/condition_summary.csv SHA-256 1cb072b4db7c8e29212ef28b97baccb66a507a7c8b8ceaad35b7e84f7976a951). - Autonomous peer review and revision: an autonomous internal peer-review and revision loop ran after the first draft; requested clarifications that could be supported by archived artifacts were incorporated (for example: explicit inclusion of the master prompt archive path and SHA-256, representative scoring JSON hashes, and clarification that the confirmatory semantics used a single shared LLM candidate per task). No human manual edits of the technical content, numbers, validator outputs, or manuscript interpretation occurred after the autonomy lock. - Technical restarts: only preregistered retries intended to recover genuine infrastructure failures were allowed and recorded in execution/execution_log.jsonl (SHA-256 a5659f3ac8ca4cf9a6b9dadaab1752af9dcebc4e7c370f0a93a234586cb55873). The run completed without infrastructure failures; there were no reruns intended to alter scientific outcomes. - Pre-lock development: pre-lock collaborative infrastructure development, rehearsal, and model/tooling experiments occurred but were explicitly excluded from final empirical claims; only the post-lock preregistered execution artifacts are used as evidence for reported results. - Reproducibility: the full artifact bundle (preregistration, task manifests, model configuration, provider traces, raw responses, validator scoring JSONs, analysis scripts and outputs) is archived and SHA-256 hashed so the exact run can be deterministically reproduced. Representative artifact paths and SHA-256 strings are included above and throughout the manuscript text. - Limitations of this run (summary): the confirmatory design used a shared LLM candidate per task so no repair loops were exercised in confirmatory episodes; the Mininet emulator and synthetic task templates limit direct production generalizability to heterogeneous vendor hardware and timing variations; the deterministic validator covers a finite rule set and may not capture every operational hazard. See the limitations section in the manuscript for full detail.

## References


[1] https://doi.org/10.1145/3656296
[2] https://doi.org/10.48550/arxiv.2310.07637
[3] https://doi.org/10.48550/arxiv.2309.05557